A solid blue vertical bar is positioned on the left side of the slide.

MuJoCo Tutorial

Table of Contents



- ✓ 1. What is MuJoCo?
 - ✓ 2. Installation
 - ✓ 3. RL Practice with MuJoCo
-

What is MuJoCo?

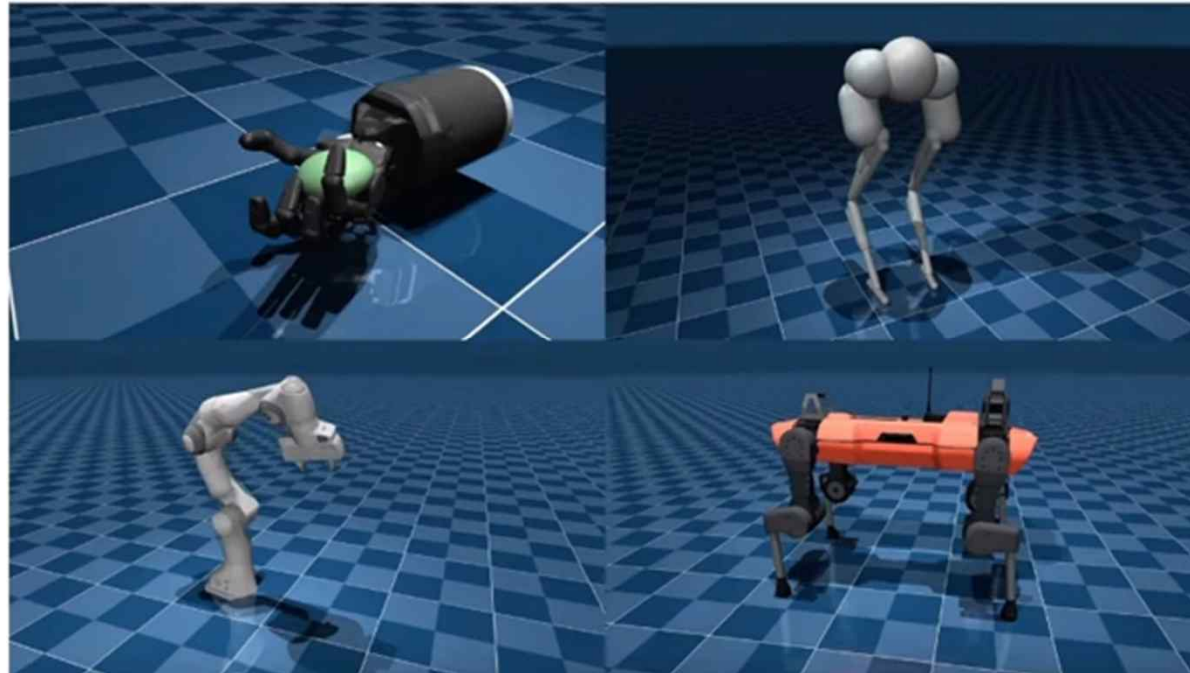


- ✓ 1. Introduction
- ✓ 2. Implementation with MuJoCo
- ✓ 3. RL Benchmarks with MuJoCo

What is MuJoCo?

✓ Introduction

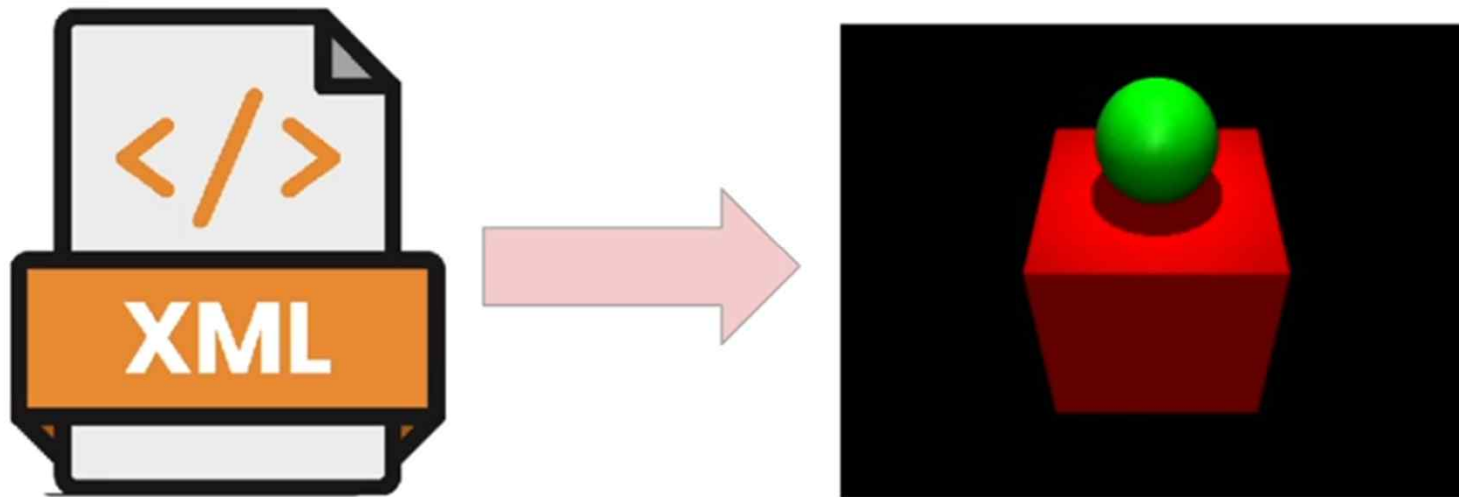
- ❑ Multi-Joint Dynamics with Contact (접촉을 포함한 다관절 동역학)
- ❑ MuJoCo is a physics engine.
- ❑ MuJoCo aims to facilitate research and development in robotics and reinforcement learning.
- ❑ MuJoCo is freely available by DeepMind in October 2021.



What is MuJoCo?

✓ Introduction

- You can defines models in XML file using the MJCF language.
- With XML file, MuJoCo constructs the models.



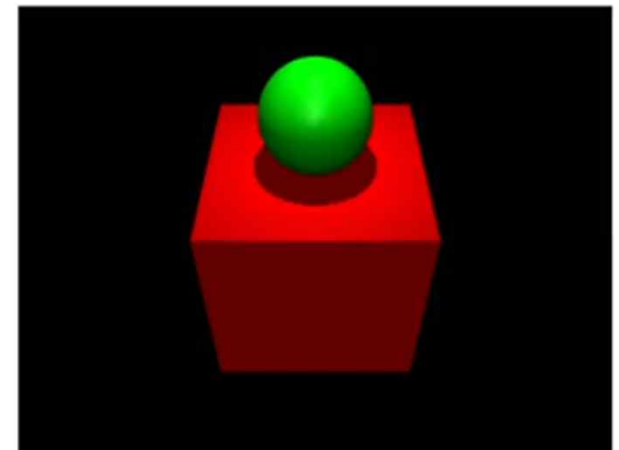
What is MuJoCo?

✓ Implementation with MuJoCo

□ Introduction

- You can define models in XML file using the MJCF language.
- With XML file, MuJoCo constructs the models.

```
xml = """  
<mujoco>  
  <worldbody>  
    <light name="top" pos="0 0 1"/>  
    <body name="example">  
      <geom name="red_box" type="box" size=".2 .2 .2" rgba="1 0 0 1"/>  
      <geom name="green_sphere" pos="0 0 .3" size=".1" rgba="0 1 0 1"/>  
    </body>  
  </worldbody>  
</mujoco>  
"""
```

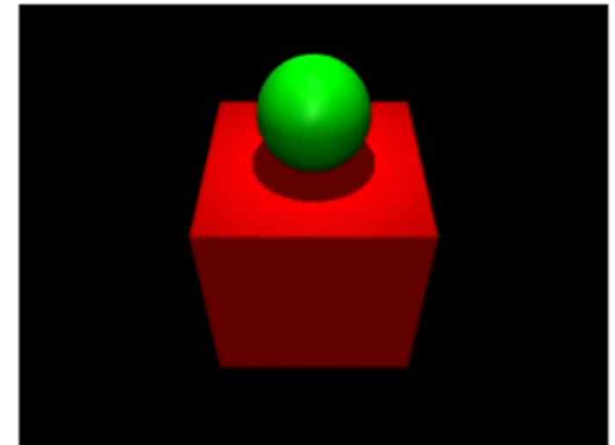


What is MuJoCo?

✓ Implementation with MuJoCo

- ❑ You can define models in XML file using the MJCF language.
- ❑ With XML file, MuJoCo constructs the models.
- ❑ You declare a worldbody first and define a body inside it.
- ❑ geom element contains name, type, pose, and rgba.
 - ❑ type: box, sphere, cylinder and so on

```
xml = ""
<mujoco>
  <worldbody>
    <light name="top" pos="0 0 1"/>
    <body name="example">
      <geom name="red_box" type="box" size=".2 .2 .2" rgba="1 0 0 1"/>
      <geom name="green_sphere" pos="0 0 .3" size=".1" rgba="0 1 0 1"/>
    </body>
  </worldbody>
</mujoco>
""
```

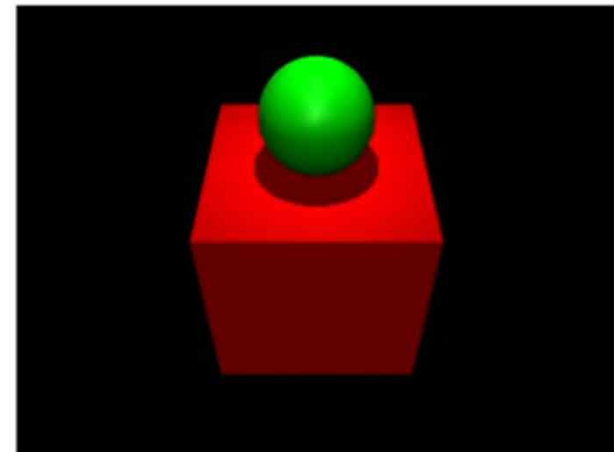


What is MuJoCo?

✓ Implementation with MuJoCo

- ❑ You can define models in XML file using the MJCF language.
- ❑ With XML file, MuJoCo constructs the models.
- ❑ For more information, <https://mujoco.readthedocs.io/en/latest/modeling.html>

```
1 import mujoco
2 import mediapy as media
3 import matplotlib.pyplot as plt
4
5 xml = """
6 <mujoco>
7   <worldbody>
8     <light name="top" pos="0 0 1"/>
9     <geom name="red_box" type="box" size=".2 .2 .2" rgba="1 0 0 1"/>
10    <geom name="green_sphere" pos="0 0 .3" size=".1" rgba="0 1 0 1"/>
11  </worldbody>
12 </mujoco>
13 """
14 # Create a MuJoCo model from the XML string
15 model = mujoco.MjModel.from_xml_string(xml)
16
17 # Create a data structure for the model
18 data = mujoco.MjData(model)
19
20 # Create a renderer for visualizing the model
21 renderer = mujoco.Renderer(model)
22
23 # Perform a forward dynamics step to update the model's state
24 mujoco.mj_forward(model, data)
25
26 # Update the scene in the renderer with the current state of the data
27 renderer.update_scene(data)
28
29 # Render the scene and display the image using mediapy
30 media.show_image(renderer.render())
```



What is MuJoCo?

✓ RL Benchmarks with MuJoCo

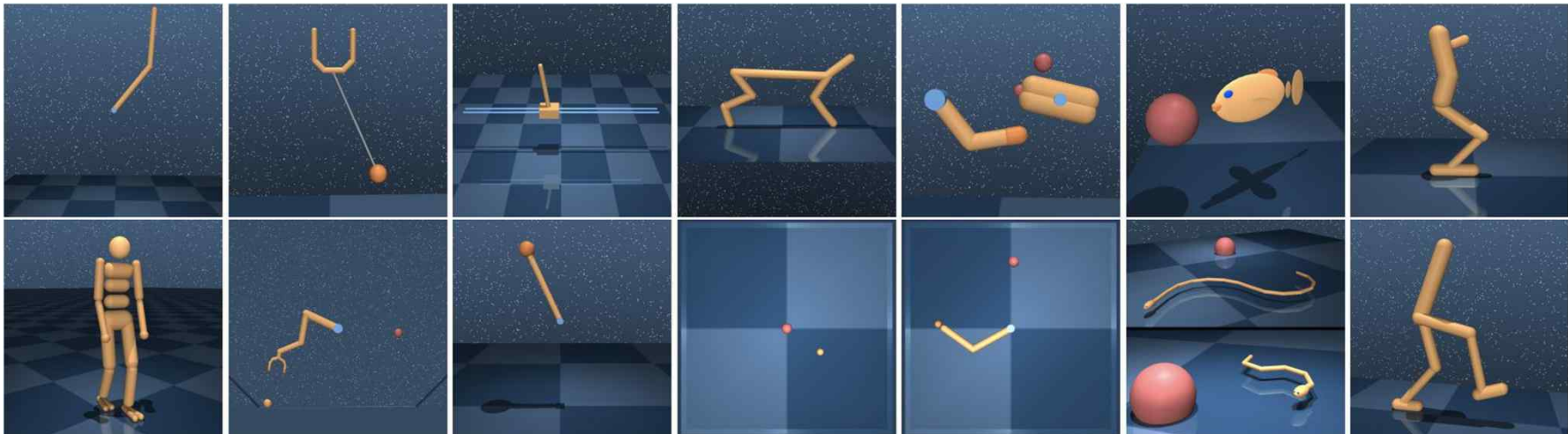
- Many RL Benchmarks are constructed based on MuJoCo.
- DMControl
- DMControl-GB
- Robosuite
- Adroit Hand
- Franka Kitchen
- LocoMuJoCo

What is MuJoCo?

✓ RL Benchmarks with MuJoCo

 DMControl

- ❑ DMControl is the Google DeepMind' s software using MuJoCo.
- ❑ DMControl contains a set of continuous control tasks for RL agents.
- ❑ You can use vector states or image states.



What is MuJoCo?

✓ RL Benchmarks with MuJoCo

□ DMControl-GB

- DMControl-GB is the benchmark for generalization of RL.
- In DMControl-GB, agents utilize images as states not manually featured vector states.
- The objective of DMControl-GB is to evaluate the generalization performance of visual RL.

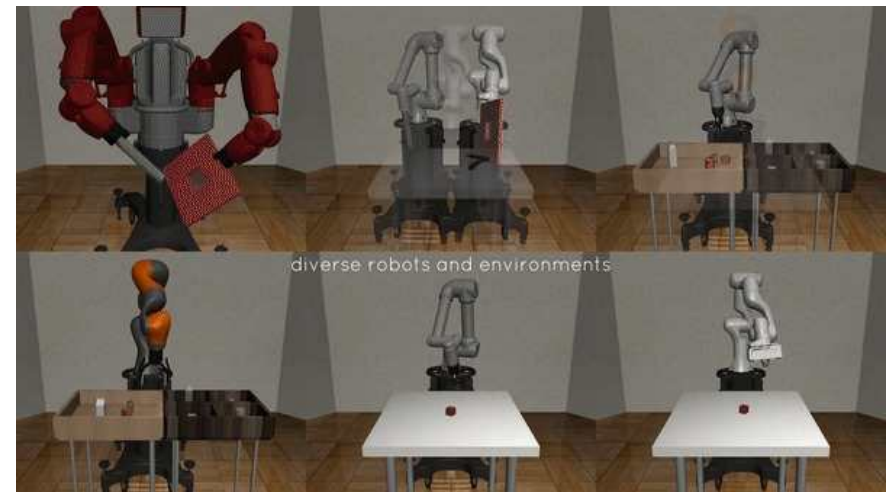
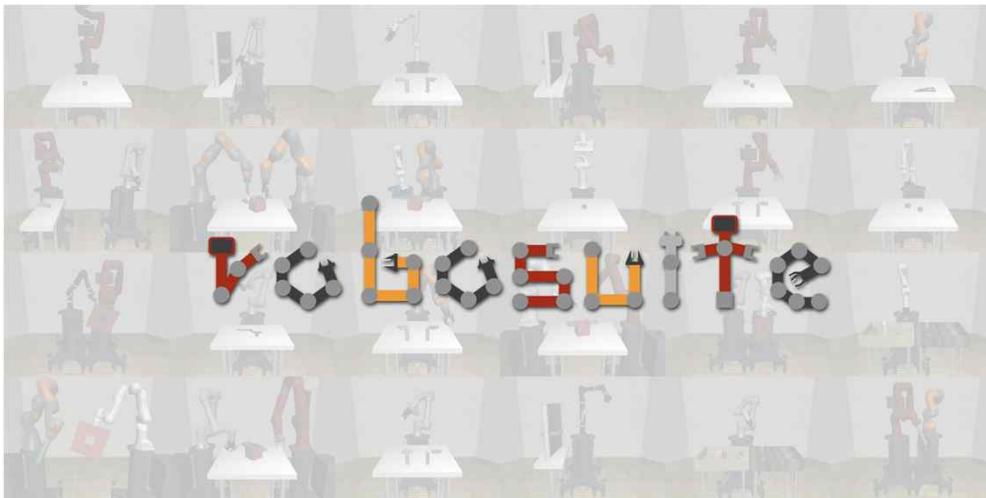


What is MuJoCo?

✓ RL Benchmarks with MuJoCo

▣ Robosuite

- ▣ Robosuite is a simulation framework aiming to facilitate research of RL and Imitation Learning for robotics.
- ▣ There are seven robot models.
- ▣ There are 9 possible tasks.
- ▣ Robosuite also provides vector state or image state.

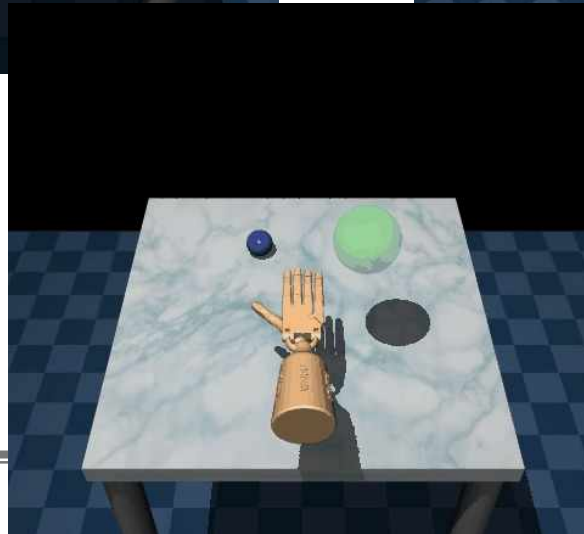
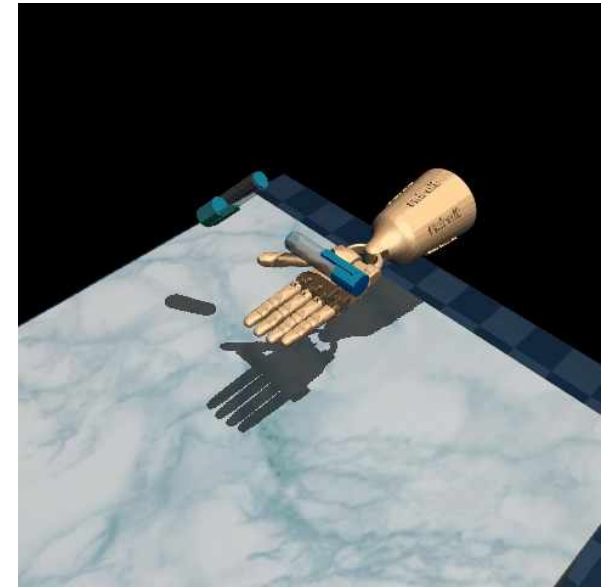


What is MuJoCo?

✓ RL Benchmarks with MuJoCo

▣ Adroit Hand

- ▣ This environments consist of a Shadow Dexterous Hand attached to a free arm.
- ▣ There are 4 possible tasks to be solved.
- ▣ These only provide vector states.



What is MuJoCo?

✓ RL Benchmarks with MuJoCo

▣ Franka Kitchen

- ▣ The Franka robot is placed in a kitchen environment containing several common household items: a microwave, a kettle, an overhead light, cabinets, and an oven.
- ▣ The goal of each task is to interact with the items.



What is MuJoCo?

✓ RL Benchmarks with MuJoCo

▣ LocoMuJoCo

- ▣ LocoMuJoCo is the benchmark specially targeted towards locomotion.
- ▣ It contains a diverse set of environments, including quadruped, bipeds, and skeleton human models.



Installation

- ✓ Step1: Download the MuJoCo library
- ✓ Step2: Create a hidden folder (.mujoco)
- ✓ Step3: Extract the library to the .mujoco
- ✓ Step4: Include lines in .bashrc file
- ✓ Step5: Run source .bashrc
- ✓ Step6: Install mujoco-py
- ✓ Step7: Make Conda Environment for Running mujoco-py

We assume that you already have Conda, torch, torchvision, torchaudio, and CUDA installed, and that you are also using Ubuntu version 18.04 or later.

In this practice, we are using torch version 1.13.1, torchvision version 0.14.1, torchaudio version 0.13.1, CUDA version 11.4, and Ubuntu 20.04

Installation

- ✓ **Step1: Download the MuJoCo library**

- ▣ https://mujoco.org/download/mujoco210-linux-x86_64.tar.gz

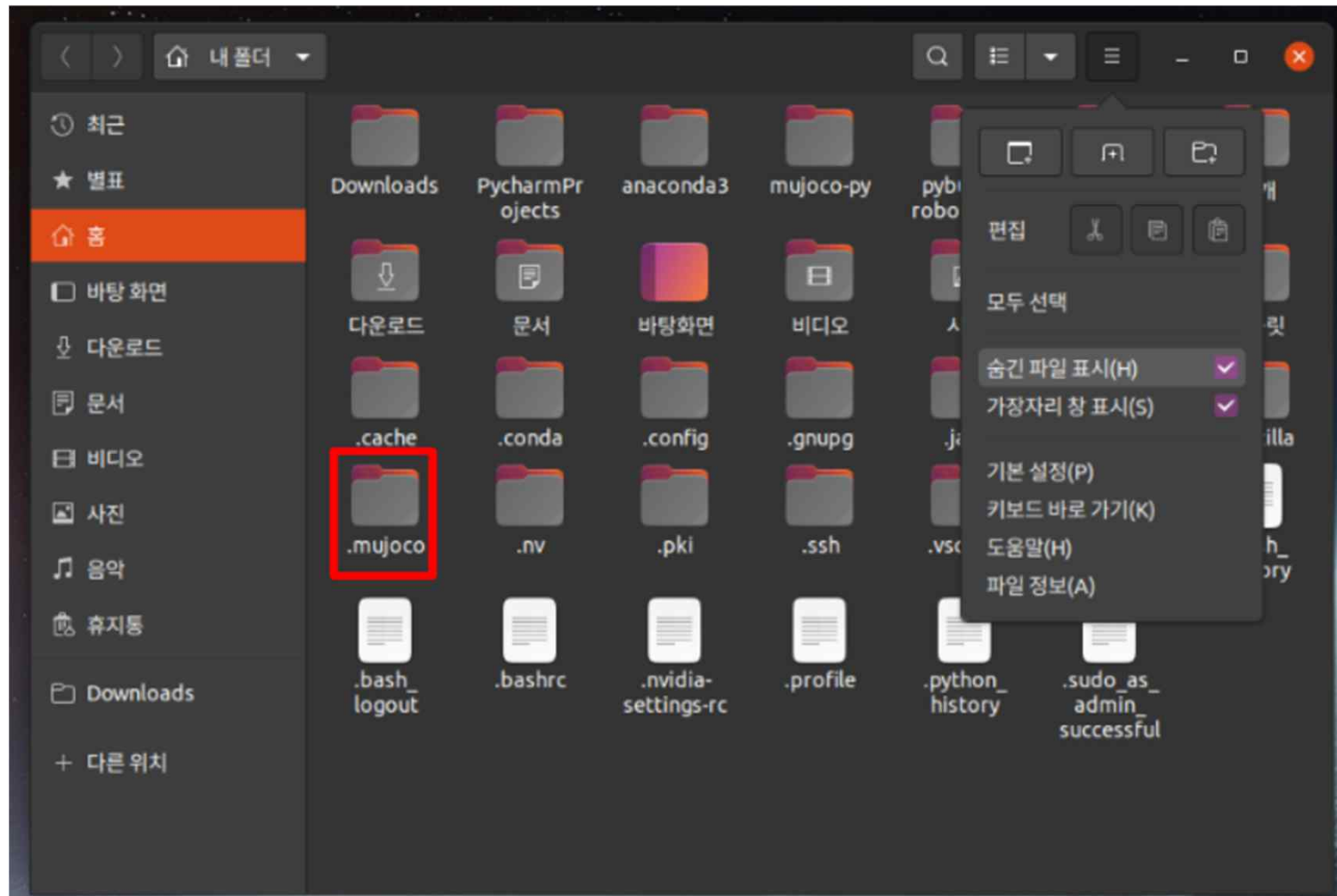


mujoco210-
linux-x86_
64.tar.gz

Installation

✓ Step2: Create a hidden folder (.mujoco)

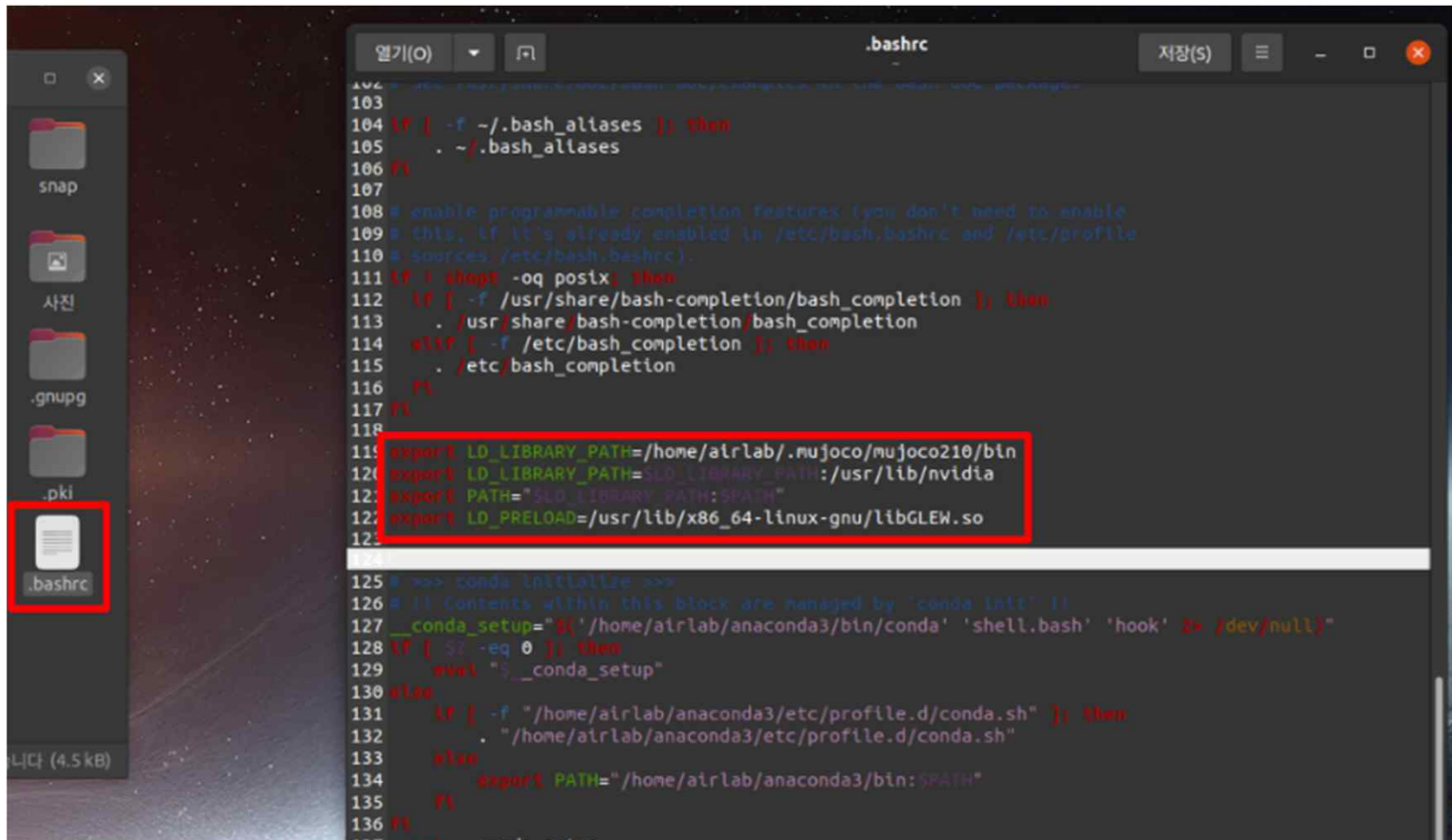
❑ `mkdir /home/$USER/.mujoco`



Installation

✓ Step4: Include lines in .bashrc file

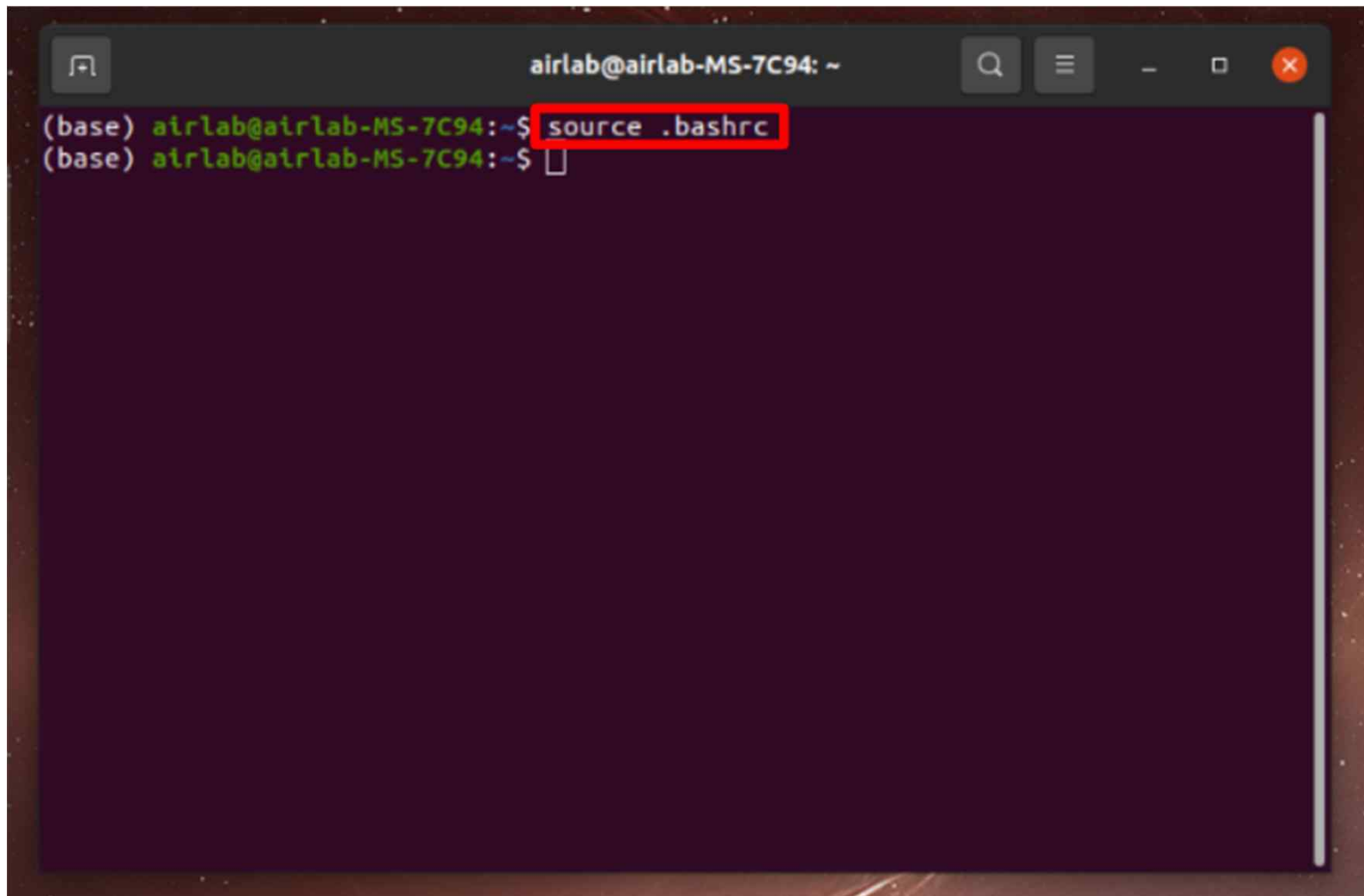
- ❑ export LD_LIBRARY_PATH=/home/\$USER/.mujoco/mujoco210/bin
- ❑ export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:/usr/lib/nvidia
- ❑ export PATH="\$LD_LIBRARY_PATH:\$PATH"
- ❑ export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libGLEW.so



```
102
103
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc)
111 if ! shopt -oq posix; then
112     if [ -f /usr/share/bash-completion/bash_completion ]; then
113         . /usr/share/bash-completion/bash_completion
114     elif [ -f /etc/bash_completion ]; then
115         . /etc/bash_completion
116     fi
117 fi
118
119 export LD_LIBRARY_PATH=/home/airlab/.mujoco/mujoco210/bin
120 export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/lib/nvidia
121 export PATH="$LD_LIBRARY_PATH:$PATH"
122 export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libGLEW.so
123
124
125 # see >>> conda initialize >>>
126 # !! Contents within this block are managed by 'conda init' !!
127 __conda_setup="('/home/airlab/anaconda3/bin/conda' 'shell.bash' 'hook' 2> /dev/null)"
128 if [ $? -eq 0 ]; then
129     eval "$__conda_setup"
130 else
131     if [ -f "/home/airlab/anaconda3/etc/profile.d/conda.sh" ]; then
132         . "/home/airlab/anaconda3/etc/profile.d/conda.sh"
133     else
134         export PATH="/home/airlab/anaconda3/bin:$PATH"
135     fi
136 fi
137
```

Installation

- ✓ Step5: Run `source .bashrc`



```
airlab@airlab-MS-7C94: ~  
(base) airlab@airlab-MS-7C94:~$ source .bashrc  
(base) airlab@airlab-MS-7C94:~$
```

A terminal window with a dark purple background and a light blue title bar. The title bar contains the text "airlab@airlab-MS-7C94: ~" and standard window control buttons (search, menu, close). The terminal shows two lines of text: "(base) airlab@airlab-MS-7C94:~\$ source .bashrc" and "(base) airlab@airlab-MS-7C94:~\$". The command "source .bashrc" is highlighted with a red rectangular box.

Installation

✓ Step6: Install mujoco-py

- ❑ `conda create --name mujoco_py python=3.7`
- ❑ `conda activate mujoco_py`
- ❑ `sudo apt update`
- ❑ `sudo apt-get install patchelf`
- ❑ `sudo apt-get install python3-dev build-essential libssl-dev libffi-dev libxml2-dev`
- ❑ `sudo apt-get install libxslt1-dev zlib1g-dev libglew1.5 libglew-dev python3-pip`
- ❑ `git clone https://github.com/openai/mujoco-py`
- ❑ `cd mujoco-py`

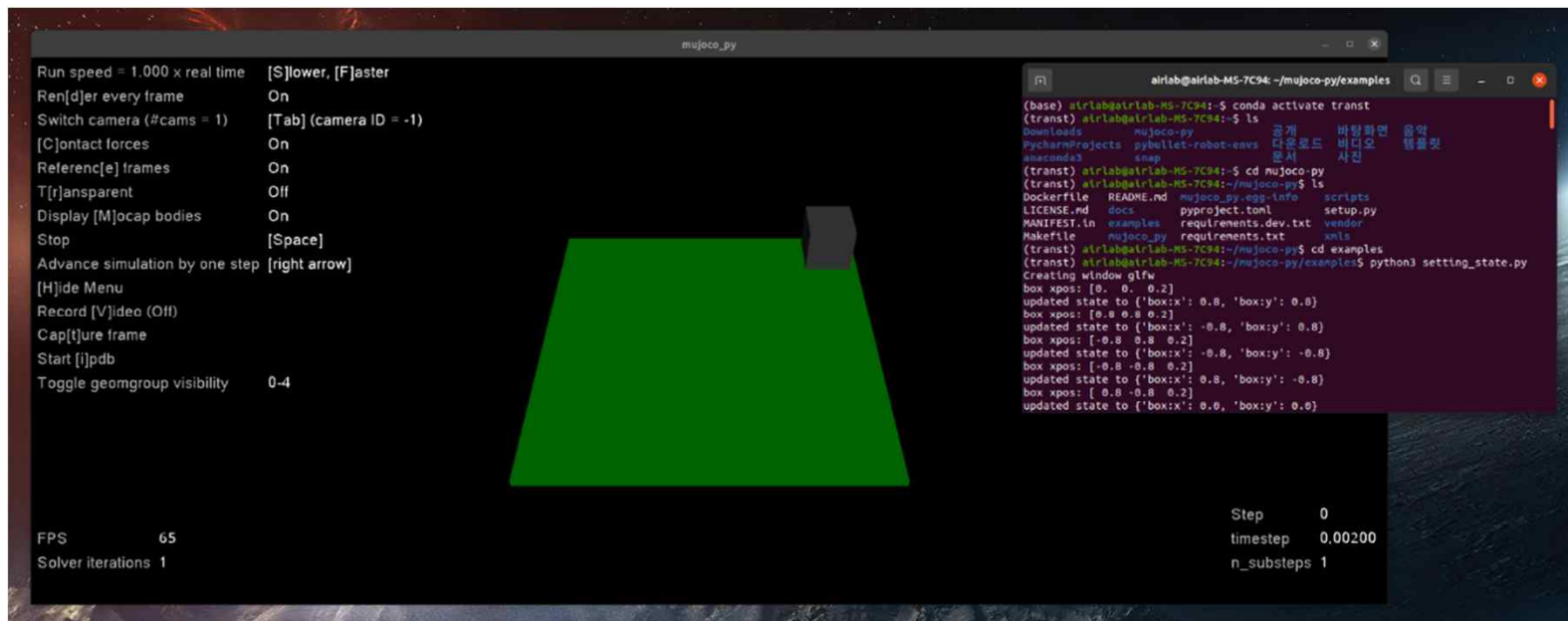
- ❑ `pip install -r requirements.txt`
- ❑ `pip install -r requirements.dev.txt`
- ❑ `pip install -e . --no-cache`

✓ Next, Reboot your Machine !!

Installation

✓ Step7: Make Conda Environment for Running mujoco-py

- ❑ conda activate mujoco_py
- ❑ cd mujoco-py
- ❑ sudo apt install libosmesa6-dev libgl1-mesa-glx libglfw3
- ❑ sudo ln -s /usr/lib/x86_64-linux-gnu/libGL.so.1/usr/lib/x86_64-linux-gnu/libGL.so
- ❑ pip3 install -U 'mujoco-py<2.2,>=2.1'
- ❑ pip install "cython<3"
- ❑ cd examples
- ❑ python3 setting_state.py



RL Practice with MuJoCo

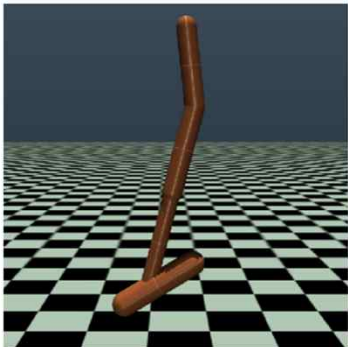
- ✓ Overview
 - ✓ Tasks
 - ✓ Code Interface
 - ✓ Codebase
 - ✓ Colab
-

RL Practice with MuJoCo

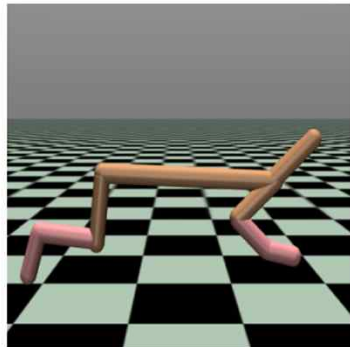
✓ Overview

- In this time, we will learn how to use the MuJoCo environments provided by openai gym for RL practice.
- Environments: Hopper, HalfCheetah, Walker2D, Ant, Humanoid

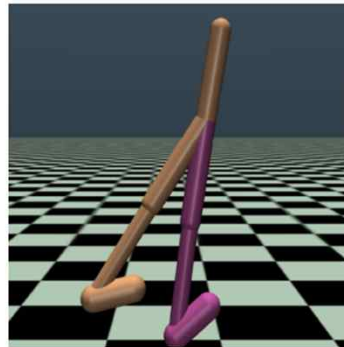
Hopper



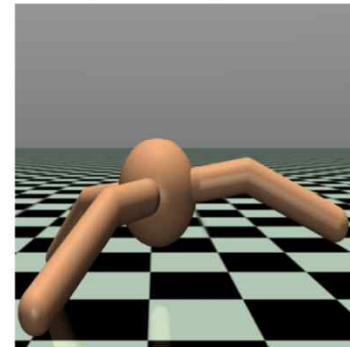
HalfCheetah



Walker2D



Ant



Humanoid

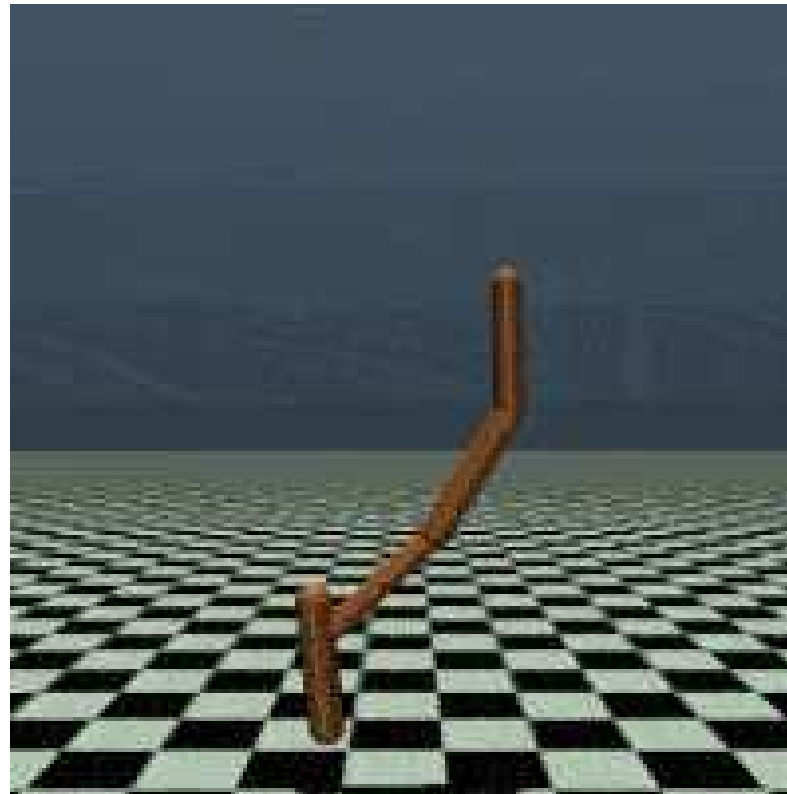


RL Practice with MuJoCo

✓ Tasks

□ Hopper

- Action Space : Box (-1.0, 1.0, (3,), float32)
- Observation Space: Box(-inf, inf, (11,), float64)
- Import: `gym.make( "Hopper-v2" )`

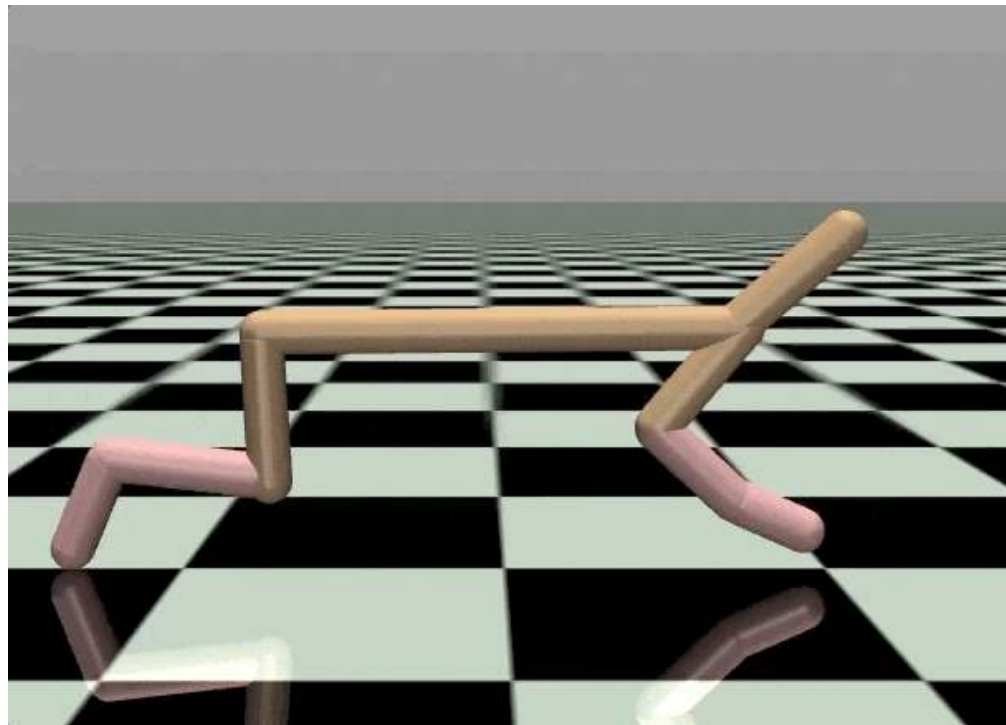


RL Practice with MuJoCo - Tasks

✓ Tasks

□ HalfCheetah

- Action Space : Box (-1.0, 1.0, (6,), float32)
- Observation Space: Box(-inf, inf, (17,), float64)
- Import: `gym.make( "HalfCheetah-v2" )`

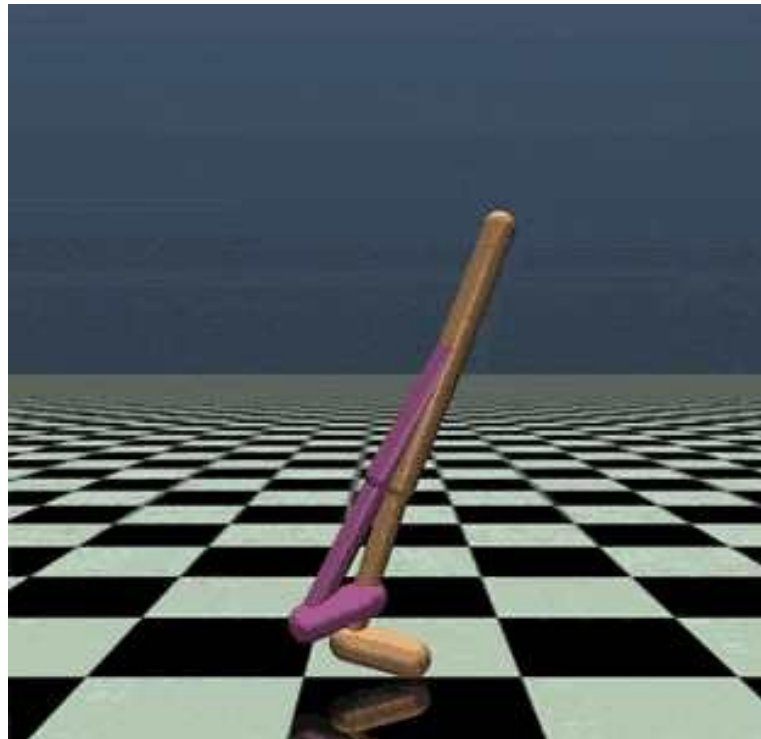


RL Practice with MuJoCo - Tasks

✓ Tasks

□ Walker2D

- Action Space : Box (-1.0, 1.0, (6,), float32)
- Observation Space: Box(-inf, inf, (17,), float64)
- Import: `gym.make( "Walker2d-v2" )`

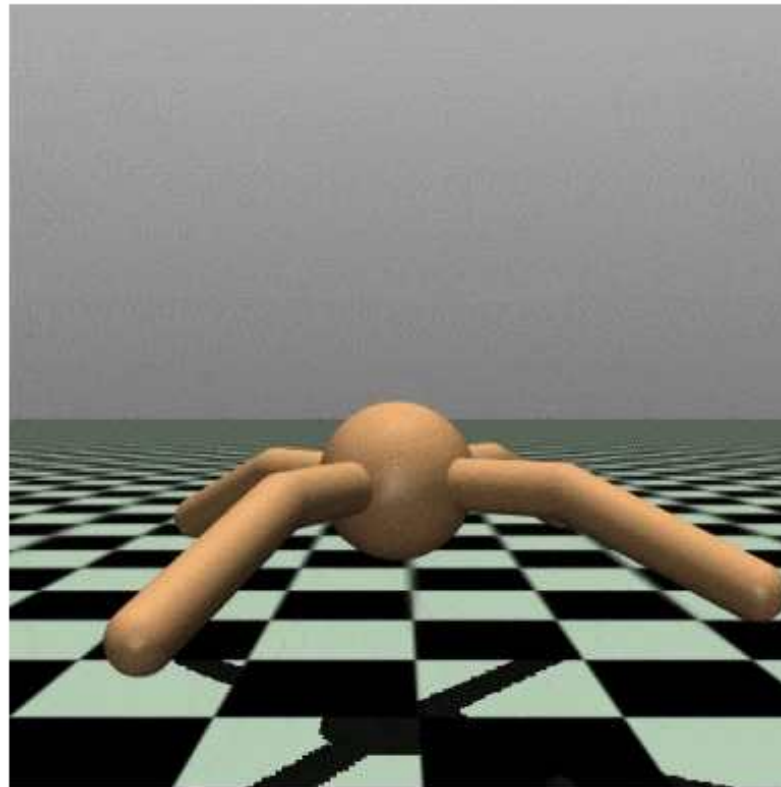


RL Practice with MuJoCo - Tasks

✓ Tasks

□ Ant

- Action Space : Box (-1.0, 1.0, (8,), float32)
- Observation Space: Box(-inf, inf, (27,), float64)
- Import: `gym.make( "Ant-v2" )`

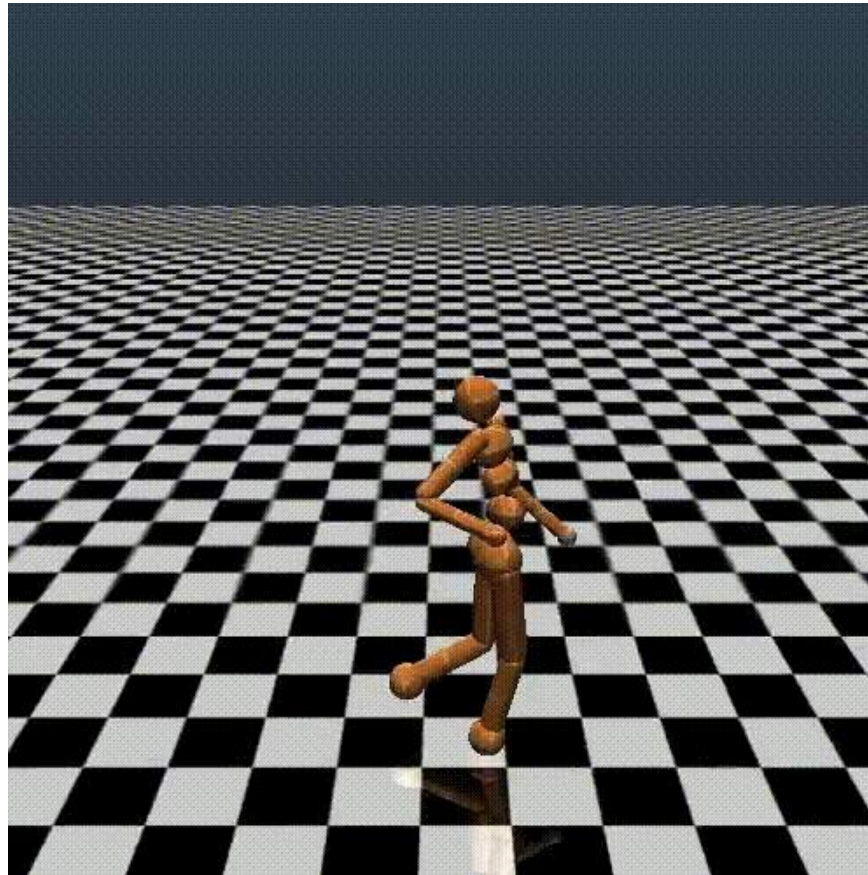


RL Practice with MuJoCo - Tasks

✓ Tasks

□ Humanoid

- Action Space : Box (-1.0, 1.0, (17,), float32)
- Observation Space: Box(-inf, inf, (376,), float64)
- Import: `gym.make( "Humanoid-v2" )`



RL Practice with MuJoCo - Code Interface

✓ Code Interface

- The interface to utilize MuJoCo
- environment is identical to
- other gym environments.

```
1 import gym
2 import time
3
4 # Create the HalfCheetah-v2 environment
5 env = gym.make('HalfCheetah-v2')
6
7 # Initialize the environment
8 observation = env.reset()
9
10 # Set the number of steps to run
11 num_steps = 1000
12
13 # Run the simulation
14 for step in range(num_steps):
15     # Select a random action
16     action = env.action_space.sample()
17
18     # Step the environment
19     observation, reward, done, info = env.step(action)
20
21
22     # If the episode is done, reset the environment
23     if done:
24         observation = env.reset()
25
26 # Close the environment
27 env.close()
```

RL Practice with MuJoCo

✓ Codebase

- We are providing the Codebase that you can run SAC in MuJoCo environments.
- Codebase: <https://github.com/AIRLABkhu/MujocoTestbed>

The screenshot shows the GitHub repository page for **MujocoTestbed** by user **TropicalPenguinn**. The repository is public and has 114 commits, 2 stars, and 1 watcher. The file list includes folders for DDPG, SAC, and TD3, and files for LICENSE, README.md, plotter.py, requirements.txt, and tools.py. The README section is visible at the bottom, titled **MujocoTestbed** and described as **MujocoTestbed of AIRLAB**. The right sidebar contains sections for About, Releases, Packages, and Contributors.

MujocoTestbed Public

Watch 1 Fork 0 Star 2

main 1 Branch 0 Tags

Go to file t Add file <> Code

TropicalPenguinn Update plotter.py e3bc4ea · last week 114 Commits

File/Folder	Commit Message	Time
DDPG	Update main.py	2 years ago
SAC	Add files via upload	2 years ago
TD3	Update main.py	2 years ago
LICENSE	Add files via upload	2 years ago
README.md	Update README.md	2 years ago
plotter.py	Update plotter.py	last week
requirements.txt	Update requirements.txt	last week
tools.py	Add files via upload	2 years ago

README MIT license

MujocoTestbed

MujocoTestbed of AIRLAB

About

MujocoTestbed

- Readme
- MIT license
- Activity
- 2 stars
- 1 watching
- 0 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 2

- TropicalPenguinn** JeongWoonLee
- AIRLABkhu** AIRLAB

RL Practice with MuJoCo

✓ Codebase

- We are providing the Codebase that you can run SAC in MuJoCo environments.
- Codebase: <https://github.com/AIRLABkhu/MujocoTestbed>
- Algorithms : SAC, DDPG, TD3
- Build
 - `git clone https://github.com/AIRLABkhu/MujocoTestbed.git`
 - `cd MujocoTestbed`
 - `pip install torch==1.13.1 torchvision==0.14.1 torchaudio==0.13.1`
 - `pip install -r requirements.txt`
- For SAC
 - `python SAC/main.py --env-name Humanoid-v2 --alpha 0.05`
- For DDPG
 - `python DDPG/main.py --env-name Humanoid-v2`
- For TD3
 - `python TD3/main.py --env-name Humanoid-v2`

RL Practice with MuJoCo

✓ Codebase

- Codebase: <https://github.com/AIRLABkhu/MujocoTestbed>



□ Plotting

- `python tools.py --root-dir run/{ENV_NAME}/{Algo}`
- `python plotter.py --task {ENV_NAME} --algo {algo1},{algo2},{algo3}` (for comparison of algorithms)
- Example
 - `python tools.py --root-dir run/Humanoid-v2/sac`
 - `python tools.py --root-dir run/Humanoid-v2/ddpg`
 - `python tools.py --root-dir run/Humanoid-v2/td3`
 - `python plotter.py --task Humanoid-v2 --algo sac,ddpg,td3`

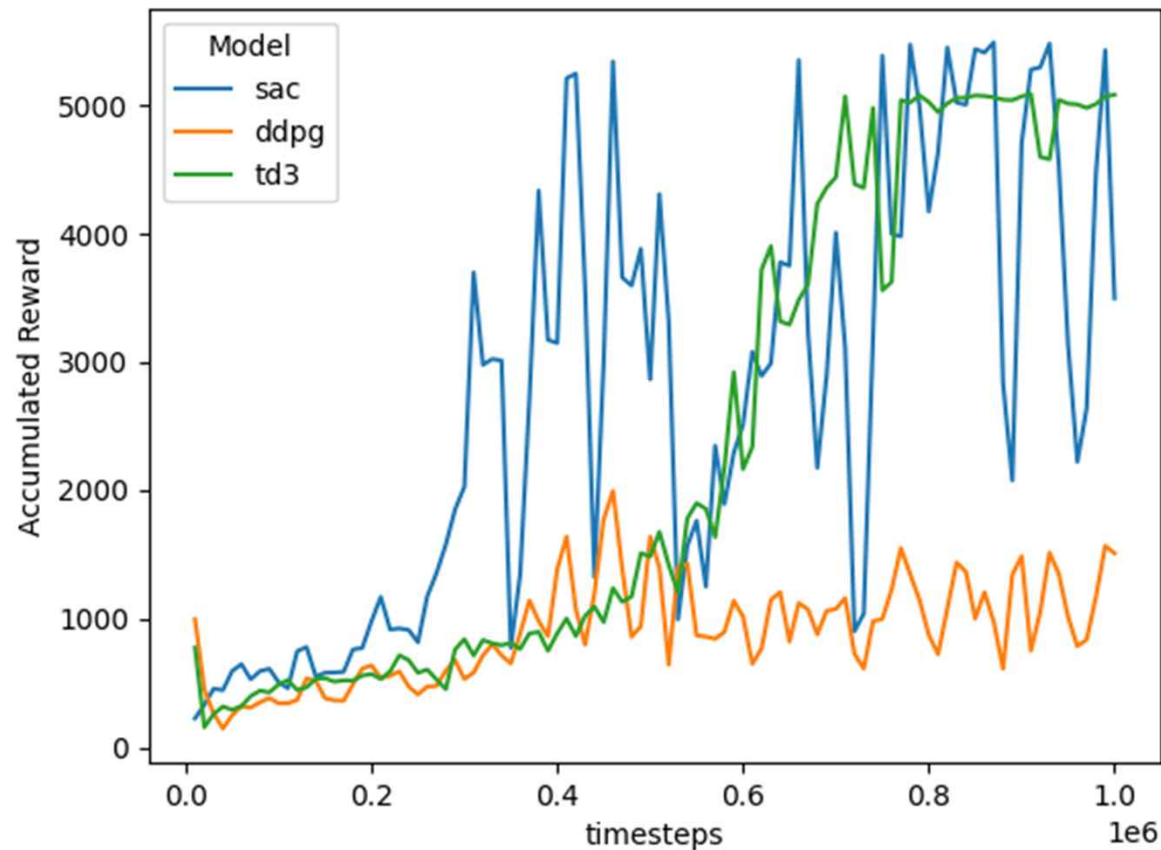
RL Practice with MuJoCo

✓ Codebase

- Codebase: <https://github.com/AIRLABkhu/MujocoTestbed>

-

- Plotting



RL Practice with MuJoCo

✓ Colab

- ❑ If you can't run MuJoCo on your PC?
- ❑ Don't worry. We will find a way. We always have.
- ❑ Let's run MuJoCo on Colab!!

The Colab logo is displayed in a large, bold, sans-serif font. The letters 'co' are yellow with a slight gradient, while the letters 'lab' are a solid orange color. The logo is centered horizontally on the slide.

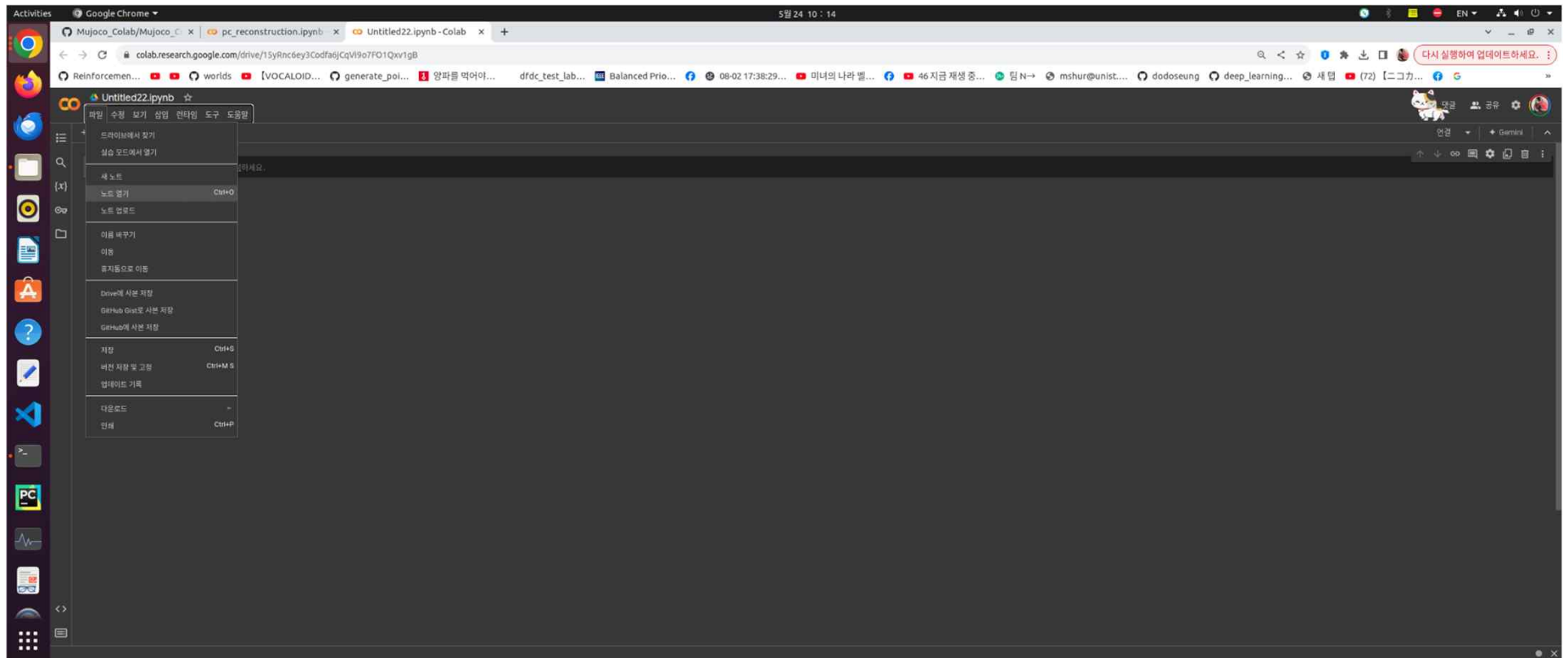
RL Practice with MuJoCo

- ✓ With this commands you can set MuJoCo on Colab
 - ❑ `!apt-get install -y \`
 - ❑ `libgl1-mesa-dev \`
 - ❑ `libgl1-mesa-glx \`
 - ❑ `libglew-dev \`
 - ❑ `libosmesa6-dev \`
 - ❑ `software-properties-common`
 - ❑ `!apt-get install -y patchelf`
 - ❑ `!git clone https://github.com/LeeJeongWoon-Workout/deep_rl_algorithm.git`
 - ❑ `!pip install gym==0.23.1`
 - ❑ `!pip install free-mujoco-py`
 - ❑ `!apt-get install -y xvfb python-opengl ffmpeg > /dev/null 2>&1`
 - ❑ `!pip install imageio==2.4.1`
 - ❑ `!pip install -U colabgymrender`
 - ❑ `!pip install mujoco`

RL Practice with MuJoCo

✓ Colab

- We provide the full ipynb file for running MuJoCo on Colab.
- Step1 : Open the Note



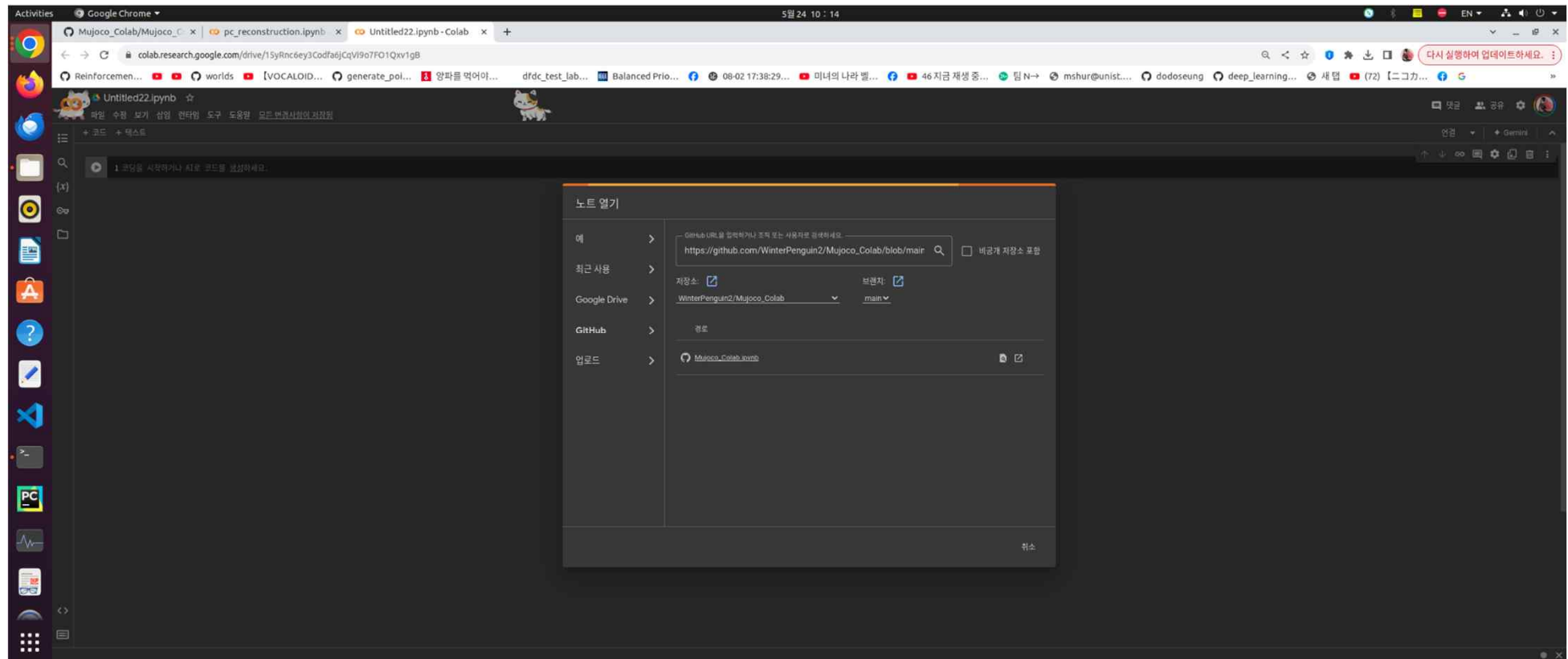
RL Practice with MuJoCo

✓ Colab

□ Step2 : Select Github and typing GitHub URL.

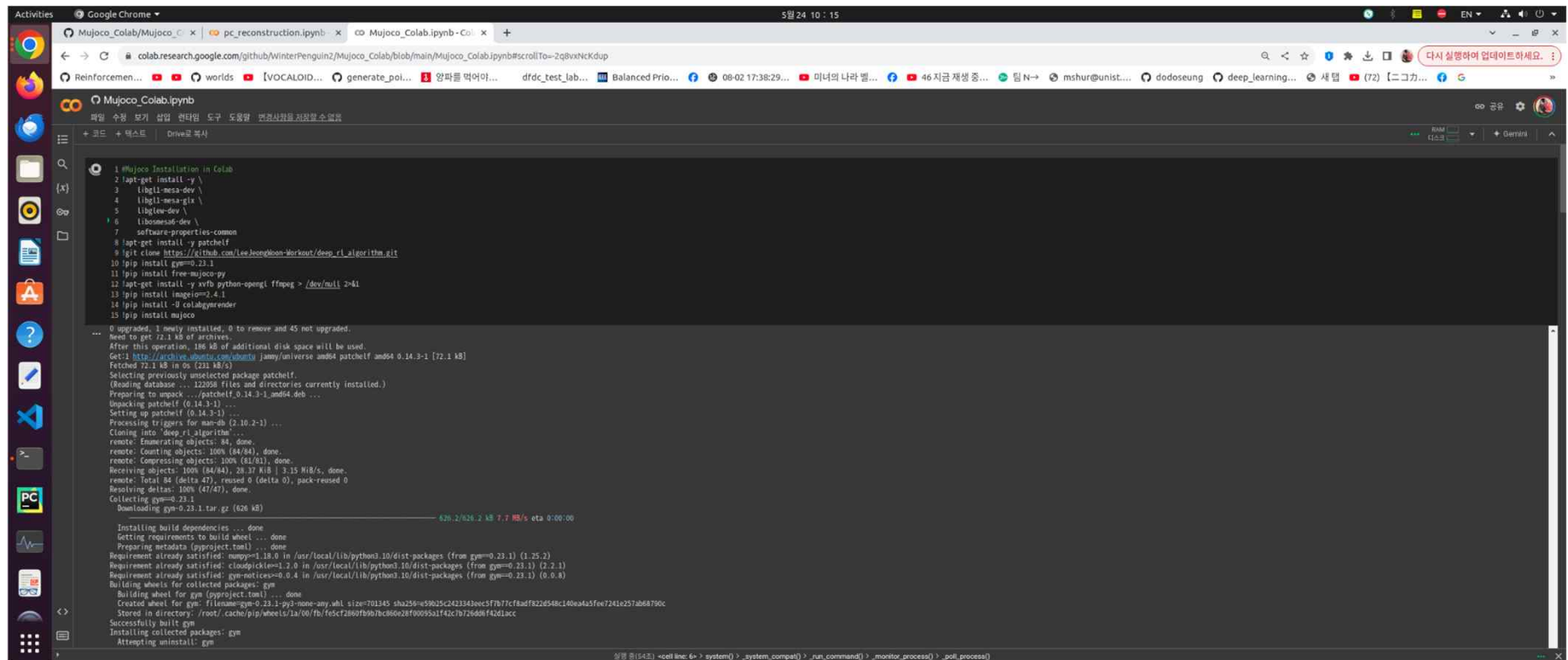
□ https://github.com/WinterPenguin2/Mujoco_Colab/blob/main/Mujoco_Colab.ipynb

□ Then, Select the ipynb file.



RL Practice with MuJoCo

- ✓ Colab
- Done!!



```
1 #Mujoco Installation in Colab
2 !apt-get install -y \
3   libgl1-mesa-dev \
4   libgl1-mesa-glx \
5   libglu1-dev \
6   libosmesa6-dev \
7   software-properties-common
8 !apt-get install -y patchelf
9 !git clone https://github.com/LeeJeonghoon-Workout/deep_rl_algorithm.git
10 !pip install gym==0.23.1
11 !pip install free-mujoco-py
12 !apt-get install -y xvfb python-opengl ffmpeg > /dev/null 2>&1
13 !pip install imageio==2.4.1
14 !pip install -U colabgameviewer
15 !pip install mujoco

...
0 upgraded, 1 newly installed, 0 to remove and 45 not upgraded.
Need to get 22.1 kB of archives.
After this operation, 186 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 patchelf amd64 0.14.3-1 [72.1 kB]
Fetched 72.1 kB in 0s (231 kB/s)
Selecting previously unselected package patchelf.
(Reading database ... 122958 files and directories currently installed.)
Preparing to unpack .../patchelf_0.14.3-1_amd64.deb ...
Unpacking patchelf (0.14.3-1) ...
Setting up patchelf (0.14.3-1) ...
Processing triggers for man-db (2.10.2-1) ...
Cloning into 'deep_rl_algorithm' ...
remote: Enumerating objects: 84, done.
remote: Counting objects: 100% (84/84), done.
remote: Compressing objects: 100% (81/81), done.
Receiving objects: 100% (84/84), 28.39 KiB | 3.15 MiB/s, done.
remote: Total 84 (delta 47), reused 0 (delta 0), pack-reused 0
Resolving deltas: 100% (47/47), done.
Collecting gym==0.23.1
  Downloading gym-0.23.1.tar.gz (626 kB)
    42%|#####| 2/236.2 kB 7.7 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: numpy<1.18.0 in /usr/local/lib/python3.10/dist-packages (from gym==0.23.1) (1.25.2)
Requirement already satisfied: cloudpickle<1.2.0 in /usr/local/lib/python3.10/dist-packages (from gym==0.23.1) (2.2.1)
Requirement already satisfied: gym-notices<0.0.4 in /usr/local/lib/python3.10/dist-packages (from gym==0.23.1) (0.0.8)
Building wheels for collected packages: gym
  Building wheel for gym (pyproject.toml) ... done
  Created wheel for gym: filename=gym-0.23.1-py3-none-any.whl size=701345 sha256=e59025c24213430ec5f7b7c7cfad822d548c40e4a5f7e7241e257db6879dc
  Stored in directory: /root/.cache/pip/wheels/1a/00/fb/fd5cf280a9fb9b7bc960e28f00905a1f42c7b726d8f4261acc
Successfully built gym
Installing collected packages: gym
Attempting uninstall: gym
```



감사합니다.
